

Guía de Presentación · POO 2026

Esta guía te muestra exactamente qué decir en cada parte de la sustentación, cómo explicar el código, y cómo responder las preguntas más comunes del profesor. Léela completa antes de presentar.

1. Cómo abrir el proyecto y arrancar

Antes de presentar — checklist rápido:

- Tener NetBeans abierto con el proyecto cargado
- Haber corrido el SQL en tu base de datos de Clever Cloud
- Haber editado **Conexion.java** con tu host, BD, usuario y password
- Correr el proyecto una vez antes (F6) para verificar que conecta

Cómo ejecutar cuando el profesor lo pida:

Click derecho en el proyecto → **Run** (o simplemente presiona F6)

La consola imprime: [OK] Conexion a la base de datos establecida.

Luego aparece el menú principal en la terminal de NetBeans.

2. Guión — qué decir al presentar

Sigue este orden. Habla con calma, muestra el código mientras explicas.

— Apertura (30 segundos)

"El proyecto se llama Domicilios Brapí. Es una aplicación de escritorio en Java para gestionar pedidos a domicilio. Se conecta a una base de datos MySQL en Clever Cloud usando JDBC, y aplica los conceptos de POO: herencia, clases abstractas, interfaces y manejo de archivos."

— La base de datos (1 minuto)

Abre el script **brapi_bd.sql** y señala:

- La tabla **persona** es la base — tiene id, nombre, email, password, rol.
- **cliente** y **operador** son tablas hijo que apuntan a persona con FK.
- **pedido** se relaciona con cliente. **itempedido** une pedido con producto.
- Todo tiene ON DELETE CASCADE, o sea, si borras una persona sus datos hijos se borran solos.

— La jerarquía de clases (1 minuto)

Abre **Persona.java** y explica:

- **Persona** es abstracta — no se puede instanciar directamente.
- Tiene el método abstracto **getRol()** que cada hijo implementa distinto.

- **Cliente extends Persona** — agrega dirección.
- **Operador extends Persona** — agrega turno.
- **Administrador extends Operador** — hereda todo y solo sobrescribe getRol().

Consejo: muestra las 3 clases abiertas en NetBeans al mismo tiempo, así el profesor ve visualmente la cadena de herencia.

— Las interfaces (45 segundos)

Abre **Exportable.java** y explica:

- La interfaz Exportable define el contrato: guardarEnArchivo() y cargarDesdeArchivo().
- La implementación concreta está en ArchivoUtil.java porque es donde vive la lógica real.
- El Administrador es quien tiene acceso a esas funciones en su menú.

— La conexión a la BD (45 segundos)

Abre **Conexion.java** y explica:

- Usé el patrón Singleton: una sola conexión para todo el programa.
- Si la conexión está cerrada o es null, la vuelve a crear automáticamente.
- Al final del programa, en Main.java, se llama Conexion.cerrar() para liberarla limpiamente.

— Los DAOs (1 minuto)

Abre **ProductoDAO.java** y explica brevemente:

- Cada entidad tiene su DAO: ProductoDAO, PersonaDAO, PedidoDAO.
- Usan PreparedStatement para evitar SQL Injection — el profesor puede preguntar esto.
- PedidoDAO usa transacciones (setAutoCommit(false)) para que el pedido y sus ítems se guarden juntos o ninguno.
- El método mapear() convierte una fila del ResultSet en un objeto Java.

— Demo en vivo (2 minutos)

Corre el programa y muestra este flujo completo:

Paso	Acción	Lo que muestra
1	Login como admin	admin@brapi.co / admin123
2	Opción 4 — Listar productos	Los 10 productos de prueba
3	Opción 2 — Mayor precio	Muestra el producto más caro
4	Cerrar sesión, login cliente	luis@gmail.com / luis123
5	Opción 1 — Hacer pedido	Agrega 2 productos, confirma
6	Opción 2 — Ver mis pedidos	El pedido recién creado
7	Login operador	carlos@brapi.co / oper123
8	Opción 6 — Cambiar estado	Cambia a EN_PROCESO

3. Preguntas frecuentes del profesor

Estas son las preguntas más típicas en sustentaciones de este tema. Practica decir las respuestas en voz alta.

■ **¿Qué es herencia y cómo la usaste?**

La herencia permite que una clase hija reutilice los atributos y métodos de una clase padre. En el proyecto, Persona es la clase base abstracta con id, nombre, email y password. Cliente y Operador extienden Persona y agregan sus propios atributos — dirección y turno respectivamente. Administrador extiende Operador, así hereda todo de ambas. Gracias a esto no repito el código de login ni de toString() en cada clase.

■ **¿Por qué Persona es abstracta?**

Porque en el sistema nunca existe una 'persona genérica' — siempre es un cliente, un operador o un administrador. Al declararla abstracta, Java no me permite hacer 'new Persona()', lo que evita errores. Además obliga a que cada subclase implemente getRol(), que es el método abstracto.

■ **¿Qué es una interfaz y para qué usaste Exportable?**

Una interfaz es un contrato: define qué métodos debe tener una clase sin implementarlos. Exportable define guardarEnArchivo() y cargarDesdeArchivo(). La lógica concreta la puse en ArchivoUtil porque es una clase utilitaria separada, y el Administrador es quien la invoca desde su menú. Esto separa la responsabilidad: el modelo no sabe de archivos, y el util no sabe de menús.

■ **¿Qué es JDBC y cómo funciona la conexión?**

JDBC es la API de Java para comunicarse con bases de datos relacionales. Usé DriverManager.getConnection() pasándole la URL de Clever Cloud con usuario y contraseña. Para no abrir múltiples conexiones innecesarias, apliqué el patrón Singleton en Conexion.java: guarda la conexión en una variable estática y solo la crea si está cerrada o es null.

■ **¿Qué es un PreparedStatement y por qué lo usaste?**

PreparedStatement es una consulta SQL parametrizada — en vez de concatenar strings uso el símbolo '?' como marcador. Esto tiene dos ventajas: previene SQL Injection porque el driver escapa los valores automáticamente, y es más eficiente porque la BD puede precompilar la consulta. Lo usé en todos los DAOs para los INSERT, UPDATE, DELETE y SELECT con parámetros.

■ **¿Qué es una transacción y dónde la aplicaste?**

Una transacción agrupa varias operaciones para que se ejecuten todas o ninguna. La usé en registrarPedido() de PedidoDAO: primero inserto el pedido en la tabla pedido, obtengo el ID generado, y luego inserto cada ítem en itempedido. Si algo falla, llamo rollback() y nada queda a medias en la BD. Al inicio desactivo el autocommit con setAutoCommit(false) y al final hago commit().

■ **¿Cómo funciona el login?**

En PersonaDAO.login() hago un SELECT a la tabla persona donde email = ? AND password = ?. Si hay resultado, leo el campo rol del ResultSet y construyo el objeto correcto: si es ADMINISTRADOR creo un Administrador, si es OPERADOR creo un Operador, etc. Luego en MenuLogin verifico el rol con un switch y redirijo al menú correspondiente. Si el SELECT no devuelve filas, retorno null y muestro error.

■ **¿Qué es el patrón DAO?**

DAO significa Data Access Object. Es un patrón donde separo todo el código SQL de las clases del modelo. En vez de que Producto sepa cómo guardarse en la BD, tengo ProductoDAO que tiene todos los métodos CRUD. Esto hace el código más organizado y fácil de mantener — si cambio la BD, solo toco los DAO, no las clases del modelo ni los menús.

■ ¿Cómo funciona el RF-08, la validación de productos vacíos?

Antes de ejecutar las opciones que dependen de tener productos — listar, buscar mayor precio, menor precio o hacer un pedido — llamo al método `existenProductos()` de `ProductoDAO`. Ese método hace un `SELECT COUNT(*) FROM producto` y retorna `true` o `false`. Si retorna `false`, muestro un mensaje de error y no ejecuto la opción. Esto cumple el RF-08.

■ ¿Cómo funciona la exportación e importación de archivos?

En `ArchivoUtil.guardarProductos()` uso `PrintWriter` para escribir cada producto en formato `id|nombre|categoria|precio`, una línea por producto. En `cargarProductos()` uso `BufferedReader` para leer línea por línea, hacer `split('|')` y crear objetos `Producto`. Antes de insertarlos verifica con `buscarPorId()` que no existan ya, así evita duplicados. Si una línea tiene formato incorrecto, la salta y avisa.

■ ¿Por qué Maven y no solo agregar el .jar manualmente?

Maven gestiona las dependencias automáticamente — solo declaro `mysql-connector-java` en el `pom.xml` y Maven lo descarga. Así no tengo que incluir el `.jar` en el proyecto ni el profesor tampoco, lo cual además cumple la recomendación del enunciado de no anexar el conector.

4. Estructura del proyecto — qué hay en cada carpeta

co.brapi.conexion

Clases: `Conexion.java`

Singleton que abre y cierra la conexión a MySQL. Es lo primero que se ejecuta en `Main`.

co.brapi.modelo

Clases: `Persona`, `Cliente`, `Operador`, `Administrador`, `Producto`, `Pedido`, `ItemPedido`, `Exportable`

Las clases de negocio puras. No saben nada de BD ni de consola. Aquí vive la herencia y las interfaces.

co.brapi.dao

Clases: `ProductoDAO`, `PersonaDAO`, `PedidoDAO`

Todo el SQL está aquí. Cada clase tiene los métodos CRUD para su entidad.

co.brapi.menu

Clases: `MenuLogin`, `MenuCliente`, `MenuOperador`, `MenuAdministrador`

La interacción con el usuario. `MenuAdministrador` extiende `MenuOperador` (herencia entre menús también).

co.brapi.util

Clases: `Entrada`, `ArchivoUtil`

Herramientas reutilizables: leer datos del teclado sin errores, y exportar/importar archivos.

5. Mapa rápido: Requisito → Código

RF	Descripción corta	Dónde está en el código
01	Registrar producto	<code>MenuOperador.registrarProducto()</code> → <code>ProductoDAO.registrar()</code>
02	Producto mayor precio	<code>MenuOperador</code> → <code>ProductoDAO.getMayorPrecio()</code>
03	Producto menor precio	<code>MenuOperador</code> → <code>ProductoDAO.getMenorPrecio()</code>

04	Listar productos	MenuOperador.listarProductos() → ProductoDAO.listarTodos()
05	Exportar productos a archivo	MenuAdministrador → ArchivoUtil.guardarProductos()
06	Cargar desde productos_brapi.txt	MenuAdministrador → ArchivoUtil.cargarProductos()
07	Confirmación al cliente	MenuCliente.realizarPedido() imprime resumen + dirección
08	Validar productos vacíos	ProductoDAO.existenProductos() → verificación antes de operar
09	CRUD entidades (admin)	MenuAdministrador — editar, eliminar productos/pedidos/usuarios
10	Pedido con múltiples productos	MenuCliente.realizarPedido() → lista de ItemPedido → PedidoDAO
11	Ver mis pedidos	MenuCliente.verMisPedidos() → PedidoDAO.listarPorCliente()
12	Ver estado del pedido	MenuCliente.consultarEstado() → PedidoDAO.verEstado()

6. Consejos para la sustentación

✓ Habla del problema primero

Antes de mostrar código, di en una frase el problema que resuelve el proyecto. 'Brapi gestionaba pedidos en papel, esto los digitaliza.' El profesor valora contexto.

✓ Señala el código mientras hablas

No leas el código en voz alta. Señala la parte relevante con el cursor y explica qué hace. Ejemplo: 'acá el PreparedStatement evita inyección SQL porque los valores van separados de la consulta.'

✓ Si no sabes algo, sé honesto

Es mejor decir 'esa parte la tengo que revisar' que inventar una respuesta incorrecta. El profesor valora honestidad mucho más que inventar.

✓ Practica la demo una vez antes

Corre el programa completo una vez con tus compañeros. Verifica que el login funciona, que el pedido se guarda y que el cambio de estado aparece.

✓ Prepara el IDE

Antes de que llegue el profesor: tener NetBeans abierto, el proyecto visible en el árbol, la terminal lista. No perder tiempo buscando archivos.

✓ Términos que el profesor probablemente usará

Herencia, polimorfismo, encapsulamiento, abstracción, JDBC, PreparedStatement, transacción, DAO, Singleton, interfaz, clase abstracta, rollback. Con la sección 3 de esta guía ya tienes respuesta para todos.